# Lab 4: State Machine

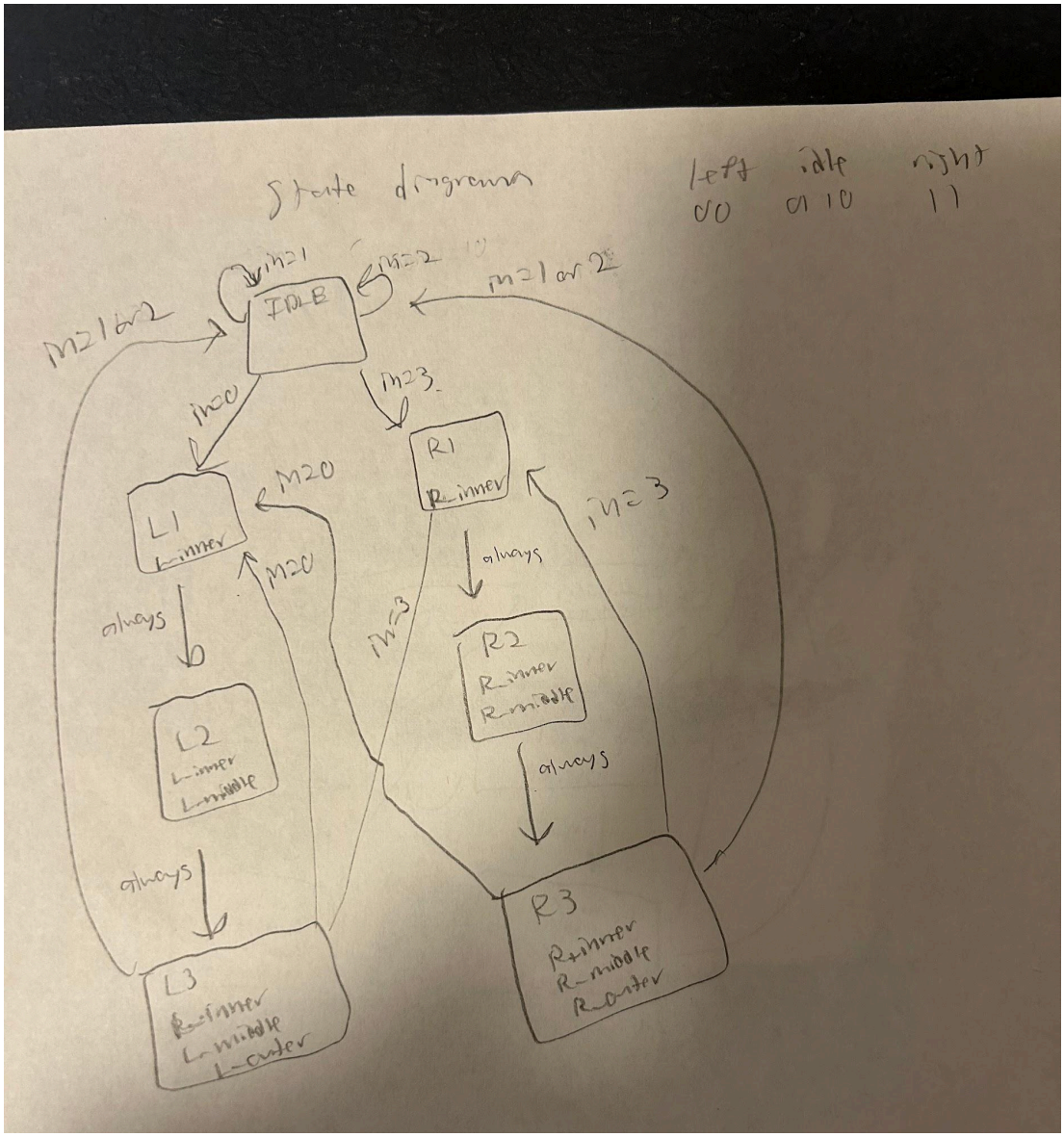
ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

# Introduction

In this lab we design, implement, and verify a synchronous finite state machine (FSM) that replicates a turn signal. Rather than a single blinking lamp, both the left and right sides use three lamps that illuminate in order from inner, middle, outer and then repeat this sequence until the signal then becomes inactive. When it comes to the driver's stalk, it is a 2-bit input that indicates straight/left/right, and also includes a clock and reset, and drives six light outputs. We will be using a moore FSM so outputs depend on state, and we assume the three-lamp sequence can complete before checking for a change to the opposite signal.

**State Machine Drawing from prelab:**




**We forgot to put this in, but if rst is asserted, we always go back to IDLE. Regardless of lightstate. When asserted, it will not complete the full three light cycle but that wasn't a requirement (just something we could assume) so we should be fine.**

# Procedure

In this lab we had started off defining a clean module interface with the clk, rst, and a 2-bit stalk input, as well as the plus six lamp outputs (either two 3-bit buses or six single bits). Next, we designed a Moore-style FSM with states Idle, Left(1, 2, 3), and Right(1, 2, 3), where each state encodes the lamp pattern and transitions advance on a timing “tick” while the corresponding stalk direction is held; any return to straight routes the machine back to Idle. To pace the sequence, we added a counter that creates a single-cycle tick every n clocks, cleanly separating timing from control. We then implemented the RTL, a synchronous state register with reset to Idle, combinational next-state logic that advances on tick according to stalk, and Moore outputs that map each state to the proper lamp bits. In the end, we built a testbench that drives clock and reset, holds straight to verify all-off behavior, asserts left and right to confirm the 1 to 2 to 3 to wrap patterns, toggles back to straight mid-sequence, and probes edge cases near tick boundaries.

Results

- What problems did you encounter while testing your steps yourself?
  **The only problems we had dealt with timing. The monitor statement only displays when changed to inputs are made, so it was hard sometimes to see what was happening when values didn’t change for repeated test cases.**

- Did any problems arise when demonstrating for the TA? What were they? Explain your thoughts on how/why these test cases escaped your own testing.
  **For the most part there were no issues. But when demoing we had to add two more idle cases for both signals 1 and 2 as well as reset state at the start.**

## SIMULATION RESULTS

**State Machine test bench:**

```
[dthornburg@linux402604 Lab4]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Oct 21 17:29 2025
Start of the light state machine test
signal=1, rst=1,                0 0 0 | 0 0 0    @                   0
signal=1, rst=0,                0 0 0 | 0 0 0    @                  10
signal=0, rst=0,                0 0 1 | 0 0 0    @                  20
signal=0, rst=0,                0 1 1 | 0 0 0    @                  30
signal=0, rst=0,                1 1 1 | 0 0 0    @                  40
signal=0, rst=0,                0 0 1 | 0 0 0    @                  50
signal=0, rst=0,                0 1 1 | 0 0 0    @                  60
signal=0, rst=0,                1 1 1 | 0 0 0    @                  70
signal=3, rst=0,                0 0 0 | 1 0 0    @                  80
signal=3, rst=0,                0 0 0 | 1 1 0    @                  90
signal=3, rst=0,                0 0 0 | 1 1 1    @                 100
signal=3, rst=0,                0 0 0 | 1 0 0    @                 110
signal=3, rst=0,                0 0 0 | 1 1 0    @                 120
signal=3, rst=0,                0 0 0 | 1 1 1    @                 130
signal=0, rst=0,                0 0 1 | 0 0 0    @                 140
signal=2, rst=0,                0 1 1 | 0 0 0    @                 150
signal=3, rst=0,                1 1 1 | 0 0 0    @                 160
signal=1, rst=0,                0 0 0 | 0 0 0    @                 170
signal=0, rst=0,                0 0 1 | 0 0 0    @                 180
signal=0, rst=0,                0 1 1 | 0 0 0    @                 190
signal=2, rst=0,                1 1 1 | 0 0 0    @                 200
signal=3, rst=0,                0 0 0 | 1 0 0    @                 210
signal=3, rst=0,                0 0 0 | 1 1 0    @                 220
signal=1, rst=0,                0 0 0 | 1 1 1    @                 230
signal=1, rst=0,                0 0 0 | 0 0 0    @                 240
signal=0, rst=0,                0 0 1 | 0 0 0    @                 250
signal=3, rst=0,                0 1 1 | 0 0 0    @                 260
signal=0, rst=0,                1 1 1 | 0 0 0    @                 270
signal=0, rst=0,                0 0 1 | 0 0 0    @                 280
signal=3, rst=0,                0 1 1 | 0 0 0    @                 290
signal=3, rst=0,                1 1 1 | 0 0 0    @                 300
signal=3, rst=0,                0 0 0 | 1 0 0    @                 310
signal=3, rst=1,                0 0 0 | 0 0 0    @                 320
signal=0, rst=0,                0 0 1 | 0 0 0    @                 330
signal=0, rst=1,                0 0 0 | 0 0 0    @                 340
signal=1, rst=0,                0 0 0 | 0 0 0    @                 350
signal=3, rst=0,                0 0 0 | 1 0 0    @                 360
signal=3, rst=0,                0 0 0 | 1 1 0    @                 370
signal=3, rst=1,                0 0 0 | 0 0 0    @                 380
signal=0, rst=0,                0 0 1 | 0 0 0    @                 390
signal=0, rst=0,                0 1 1 | 0 0 0    @                 400
signal=0, rst=1,                0 0 0 | 0 0 0    @                 410
signal=1, rst=0,                0 0 0 | 0 0 0    @                 420
signal=2, rst=0,                0 0 0 | 0 0 0    @                 460
End of Light Test
$finish called from file "tb.v", line 119.
$finish at simulation time                 500
           V C S   S i m u l a t i o n   R e p o r t
Time: 500
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Tue Oct 21 17:29:55 2025
[dthornburg@linux402604 Lab4]$
```

# Conclusion

**Overall, in this lab we created, designed, and implemented a moore FSM state machine with seven states: 3 for left, 3 for right, and 1 for idle. We tested all functions with all signals going both left and right. We had added an extra two reset cases at that start as well two idle states for one signal one and another for signal two. This state machine could be used to replicate the total outcome of an automotive turn signal.**

# Appendix

## CODE

```
module turn_signal(
        input clk, rst,
        input [1:0] signal,
        output reg L_Inner, L_Middle, L_Outer, R_Inner, R_Middle, R_Outer
);
        parameter LEFT  = 2'b00;
        parameter RIGHT = 2'b11;
        parameter IDLE1 = 2'b01;
        parameter IDLE2 = 2'b10;

        parameter StateIDLE = 3'b000;
        parameter StateL1   = 3'b001;
        parameter StateL2   = 3'b010;
        parameter StateL3   = 3'b011;
        parameter StateR1   = 3'b100;
        parameter StateR2   = 3'b101;
        parameter StateR3   = 3'b110;

        reg [2:0] curr_state, next_state;
        wire straight;
        wire go_left;
        wire go_right;
        assign straight = (signal == IDLE1) || (signal == IDLE2);
        assign go_left  = (signal == LEFT);
        assign go_right = (signal == RIGHT);

        always @(posedge clk) begin
                if (rst) curr_state <= StateIDLE;
                else curr_state <= next_state;
        end

        always @(*) begin
                // Default outputs
                L_Inner  = 1'b0;
                L_Middle = 1'b0;
                L_Outer = 1'b0;
                R_Inner  = 1'b0;
                R_Middle = 1'b0;
                R_Outer = 1'b0;

                // default next state
                next_state = curr_state;

                case (curr_state)
```

```
                        //IDLE Seq
                        StateIDLE: begin
                                if (go_left) next_state = StateL1;
                                else if (go_right) next_state = StateR1;
                                else next_state = StateIDLE;
                        end

                        //LEFT Seq
                        StateL1: begin
                                L_Inner = 1'b1;
                                next_state = StateL2;
                        end
                        StateL2: begin
                                L_Inner = 1'b1;
                                L_Middle = 1'b1;
                                next_state = StateL3;
                        end
                        StateL3: begin
                                L_Inner = 1'b1;
                                L_Middle = 1'b1;
                                L_Outer = 1'b1;
                                if (go_left) next_state = StateL1;
                                else if (go_right) next_state = StateR1;
                                else next_state = StateIDLE;
                        end

                        //RIGHT Seq
                        StateR1: begin
                                R_Inner = 1'b1;
                                next_state = StateR2;
                        end
                        StateR2: begin
                                R_Inner = 1'b1;
                                R_Middle = 1'b1;
                                next_state = StateR3;
                        end
                        StateR3: begin
                                R_Inner = 1'b1;
                                R_Middle = 1'b1;
                                R_Outer = 1'b1;
                                if (go_right) next_state = StateR1;
                                else if (go_left) next_state = StateL1;
                                else next_state = StateIDLE;
                        end

                        default: begin
                                next_state = StateIDLE;
                        end
                endcase
        end
endmodule
```

```
module testbench();
```

```
reg [1:0] signal;
reg clk, rst;
wire L_Inner, L_Middle, L_Outer, R_Inner, R_Middle, R_Outer;

turn_signal dut(clk, rst, signal, L_Inner, L_Middle, L_Outer, R_Inner, R_Middle,
R_Outer);

initial clk=1;
initial rst=0;
always #5 clk = ~clk;

initial begin
$display("Start of the light state machine test");
$monitor("signal=%d, rst=%d, \t    %d %d %d | %d %d %d \t @%d", signal, rst, L_Outer,
L_Middle, L_Inner, R_Inner, R_Middle, R_Outer, $time); //always displays at posedge of
clk
//ALSO ADD RST tests!!
signal=2'b01; rst=1;
#10
signal=2'b01; rst=0;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b00;
#10
signal=2'b10;
#10
```

```
signal=2'b11;
#10
signal=2'b01;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b10;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b01;
#10
signal=2'b01;
#10
signal=2'b00;
#10
signal=2'b11;
#10
signal=2'b00;
#10
signal=2'b00;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11; rst=1;
#10
signal=2'b00; rst=0;
#10
signal=2'b00; rst=1;
#10
signal=2'b01; rst=0;
#10

signal=2'b11;
#10
signal=2'b11;
#10
signal=2'b11; rst=1;
#10
signal=2'b00; rst=0;
#10
signal=2'b00;
```

```
#10
signal=2'b00; rst=1;
#10
signal=2'b01; rst=0;
#10
signal=2'b01; rst=0;
#10
signal=2'b01; rst=0;
#10
signal=2'b01; rst=0;
#10
signal=2'b10; rst=0;
#10
signal=2'b10; rst=0;
#10
signal=2'b10; rst=0;
#10
signal=2'b10; rst=0;
#10
$display("End of Light Test");
$finish;

end

endmodule
```